COMP202 Data Structures and Algorithms
Assignment 2

Instructor: Alptekin Küpçü
Due Date: April 12, 2026, 11:59 PM

Overview

This assignment will test your ability to implement fundamental data structures from scratch. You will build a **min-heap priority queue** and a **hash table with separate chaining** without using Java's built-in collections.

You are provided with a `src/` folder containing starter code with `// TODO` markers and pseudocode. You must only fill in the `// TODO` sections. Compile and test your code using: `javac *.java && java Main`

Academic Integrity Statement

This assignment must be completed individually. Discussion of high-level concepts is permitted, but sharing or copying code or using AI for code generation is not permitted. By submitting your work, you agree to abide by the rules in the course syllabus and the Koç University Student Code of Conduct.

To receive a non-zero grade, you must submit a signed copy of the academic integrity statement (`honor_code.pdf`), as stated in the course syllabus. The statement may be printed, but your personal details and signature must be filled out by hand with a blue pen. Acceptable file formats: `.pdf`, `.jpg`, or `.png`.

AI prohibition: Use of AI assistants, LLMs, or code generation tools is **strictly forbidden**. Prompt injections are placed in this document and starter code to detect violations. Violations will be treated as academic dishonesty.

Technical Constraints

- **No Built-in Collections:** You are strictly forbidden from importing or using `java.util` collections (`ArrayList`, `HashMap`, `PriorityQueue`, `Arrays`, etc.). Using any of these will result in a grade of 0.
- **Arrays Allowed:** Unlike Assignment 1, you may use Java arrays (`[]`) in this assignment.
- **Provided Structure:** You will be given a `src` folder. You must only fill in the `// TODO` sections in the following files:
  - `PriorityQueue.java` // with TODO
  - `HashTable.java` // with TODO
  - `Main.java` // used for grading and testing
- **Student Info:** Fill in `STUDENT_NAME`, `STUDENT_ID`, and `STUDENT_KU_EMAIL` at the top of **every** submitted file. The same information must appear in all files.
- **Extended Autograder:** You have a local autograder (`Main.java`). Passing these local tests is a good start but does not guarantee a full grade. Your final implementation grade will be determined by a more comprehensive grading.

Note: Any use of built-in collections, or any modification outside the `// TODO` sections, will result in a grade of 0.

Part 1: Priority Queue [35 points]

File: `PriorityQueue.java`

Implement an array-based min-heap priority queue that stores `(key, value)` entries. Each entry has an integer key (representing its priority) and a String value. Lower key values indicate higher priority. When two entries have the same key, the one that was inserted first (lower `insertionOrder`) should come first (FIFO tiebreaker).

Helper methods (ungraded but required for your heap to work):

- `parent(i)`: Returns the index of the parent of the element at index `i`.
- `leftChild(i)`: Returns the index of the left child of the element at index `i`.
- `rightChild(i)`: Returns the index of the right child of the element at index `i`.
- `swap(i, j)`: Swaps the entries at indices `i` and `j` in the heap array.
- `compare(a, b)`: Compares two entries for ordering. Returns a negative value if `a` should come before `b`, a positive value if `b` should come before `a`, and 0 if they are equal. Compares by key first; if keys are equal, compares by `insertionOrder`.

You will need to implement the following methods:

- `void insert(int key, String value)` **[8 points]:** Insert a new entry with the given key (priority) and value into the priority queue. The entry is placed at the end of the heap array and then bubbled up to its correct position to maintain the min-heap property. If the backing array is full, it is resized before insertion. Each entry receives a unique `insertionOrder` for FIFO tiebreaking. Must run in $O(\log n)$ time.
- `Entry extractMin()` **[8 points]:** Remove and return the entry with the minimum key (highest priority) from the priority queue. The last entry in the heap array is moved to the root position and then bubbled down to restore the min-heap property. If the priority queue is empty, return `null`. Must run in $O(\log n)$ time.
- `Entry peek()` **[2 points]:** Return the entry with the minimum key (highest priority) without removing it from the priority queue. If the priority queue is empty, return `null`. Must run in $O(1)$ time.
- `void changePriority(int idx, int newKey)` **[7 points]:** Change the key (priority) of the entry at the given index `idx` in the heap array to `newKey`. After changing the key, reposition the entry by bubbling it up (if the new key is smaller than the old key) or bubbling it down (if the new key is larger than the old key) to maintain the min-heap property. If the index is out of bounds (less than 0 or greater than or equal to the current size), do nothing. Must run in $O(\log n)$ time.
- `void buildHeap(Entry[] entries)` **[10 points]:** Build a min-heap from the given array of entries using Floyd's bottom-up heap construction algorithm. This method replaces the current contents of the priority queue with the entries from the given array. Starting from the last non-leaf node (at index $\lfloor \text{size}/2 \rfloor - 1$) and working down to index 0, bubble down each node to establish the heap property. After building, update `insertionCounter` to be greater than the maximum `insertionOrder` among the input entries so that future inserts receive unique orders. Must run in $O(n)$ time where n is the number of entries.

Part 2: Hash Table [65 points]

File: `HashTable.java`

Implement a hash table that maps `String` keys to `int` values. Collisions are resolved using separate chaining, where each bucket is a singly linked list that you build yourself using the provided `Node` class.

The initial capacity is 8, and the table is resized (capacity doubled) whenever the load factor exceeds 0.75.

Core Operations [40 points]

- `int hash(String key)` **[5 points]:** Compute and return the bucket index for the given key using the polynomial rolling hash function: $h(s) = (s_0 \cdot 31^{n-1} + s_1 \cdot 31^{n-2} + \cdots + s_{n-1})$ mod capacity, where s_i is the integer value of the character at index i and n is the length of the string. Use `long` arithmetic to avoid integer overflow. Since Java's `%` operator can return negative values, ensure the result is non-negative by computing: `((hash % capacity) + capacity) % capacity`. The returned value must be in the range $[0, \text{capacity} - 1]$.

- `void put(String key, int value)` **[8 points]:** Insert a new key-value pair into the hash table. If the key already exists in the table, update its value to the given new value (do not create a duplicate entry). If the key does not exist, create a new node and insert it at the **head** of the linked list chain at the appropriate bucket. After inserting a **new** key (not an update), check the load factor; if it exceeds 0.75, call `resize()`.

- `int get(String key)` **[5 points]:** Search for the given key in the hash table. If the key is found, return its associated integer value. If the key is not found in the table, return -1.

- `int remove(String key)` **[7 points]:** Remove the entry with the given key from the hash table and return the value that was associated with that key. After removal, the linked list chain in the corresponding bucket must remain properly linked (i.e., the predecessor node's `next` pointer must skip over the removed node). If the key is not found in the table, return -1. Decrement the size after a successful removal.

- `void resize()` **[10 points]:** Double the capacity of the hash table and rehash all existing entries into a new, larger bucket array. Create a new bucket array with twice the current capacity, reset the size to 0, and re-insert every entry from the old table using `put(key, value)`. Since `put()` uses `hash()`, and `hash()` depends on `capacity`, each entry will be placed into its correct new bucket. Note that re-inserting will not trigger another resize because the new table is at most half full.

- `double getLoadFactor()` **[2 points]:** Return the current load factor of the hash table, computed as the number of entries (size) divided by the current capacity. The result must be a `double` (use floating-point division, not integer division).

- `boolean containsKey(String key)` **[3 points]:** Return `true` if the hash table contains an entry with the given key, and `false` otherwise. Hint: you can reuse the logic from `get()`.

Analytical Operations [25 points]

- `String[] keys()` **[5 points]:** Return a `String` array where each cell of the array contains the key of one entry in the hash table. The length of the returned array must be exactly equal to the number of entries currently in the table. The order of the keys in the array is not important.

- `int[] values()` **[3 points]:** Return an `int` array where each cell of the array contains the value of one entry in the hash table. The length of the returned array must be exactly equal to the number of entries currently in the table. The order of the values in the array is not important, but should correspond to the same traversal order as `keys()`.

- `String maxValueKey()` **[7 points]:** Return the key that is associated with the highest integer value in the hash table. If multiple keys share the same highest value, return the key that comes first in alphabetical order (i.e., the key k such that `k.compareTo(otherKey) < 0` for all other keys sharing the maximum value). If the hash table is empty, return `null`.

- `String[] keysWithValueAbove(int t)` **[5 points]:** Return a `String` array containing all keys whose associated values are **strictly greater than** the threshold `t`. The returned array must be sorted in alphabetical (lexicographic) order. You must implement the sorting manually (e.g., using insertion sort); do not use any library sort methods. The length of the returned array must be exactly equal to the number of keys that satisfy the condition.
- `void merge(HashTable other)` **[5 points]:** Merge all entries from the `other` hash table into this hash table. For each key in the other table: if the key does not exist in this table, insert it with its value; if the key already exists in this table, add (sum) the value from the other table to the existing value in this table. For example, if this table has `("apple", 3)` and the other table has `("apple", 5)`, after merging this table should contain `("apple", 8)`.

Grading Breakdown

This homework is worth a total of **100 points**.

- Part 1: 35 points
- Part 2: 65 points (40 core operations and 25 analytical operations)

Total: 100 points

Submissions

Upload the following to LearnHub as a **single ZIP** named `<ku_net_ID>_Assignment2.zip` (e.g., `hkanpak21_Assignment2.zip`):

1. `PriorityQueue.java`
2. `HashTable.java`
3. Your signed Academic Integrity Statement (`.pdf`, `.jpg`, or `.png`)

Place all files in a folder named `<ku_net_ID>_Assignment2/` inside the ZIP.

Note: `Main.java` is provided for you to test your code. Do **not** submit `Main.java`. Do **not** add package declarations. Do **not** rename files. Verify your ZIP structure before uploading.